SpExplAIn: Trustworthy and Interpretable Space Explanations of Neural Network Classifiers

Tomáš Kolárik[1], Faezeh Labbaf[1], Fabrizio Leopardi[1], Grigory Fedyukovich[1,3], Michael Wand[2], Natasha Sharygina[1]

[1] University of Lugano (USI), Lugano, Switzerland
[2] SUPSI, IDSIA, Lugano, Switzerland
[3] Florida State University, United States of America

Abstract. Neural network decisions in high-stakes domains call for explanations that are both trustworthy and understandable. Existing methods often provide either intuitive but approximate accounts or formally guaranteed explanations restricted to individual features. We present SpExplAIn, a framework for computing and interpreting sound space explanations of neural network classification. Its proof-based computation generalizes a sample into a logical formula that can capture relations among input features. A separate interpreter reuses this formula to extract and visualize aspects of interest without recomputing the explanation. We evaluate the integrated tool on medical, financial, and image-recognition tasks using fully connected and convolutional networks.

1 Introduction

Neural networks (NNs) increasingly support decisions in domains where errors are costly, such as a physician's assessment of a patient's heart-attack risk [29]. The network classifies the patient as high-risk, and the physician wants to know *why*: Which combinations of the patient's characteristics, such as age, cholesterol, or blood pressure, lead to this assessment? Would it still hold if the cholesterol were lower but the patient older? And for which other patients does the same reasoning apply? Similarly, a loan banker explaining a rejected application [39] needs to know how the requested amount relates to the applicant's income; in the EU, certain high-risk automated decisions are subject to explanation-related legal requirements [20]. To be useful in such decisions, explanations must be both *trustworthy* – formally guaranteed to hold for the network rather than estimated from samples, and understandable to users without expertise in formal methods.

Many existing explainability tools do not meet both needs. Popular tools [4, 40, 52, 53, 57] render easily readable feature attributions or heatmaps, but generally do not establish formal sufficiency guarantees: the behavior outside the sampled data remains unknown, and the explanations may even be logically inconsistent [44]. Several logic-based tools [3, 8, 60, 61] compute provably correct sufficient conditions for the classification, but restrict their form by constraining input features independently or by limiting the neighborhood of the sample. They therefore primarily express feature-wise conditions rather than relations among features. These restricted forms cannot directly capture relations between features, like the trade-off between age and cholesterol in the assessment above,

and leave little room for interpretation. Moreover, in iterative feature-wise computation, solver queries grow with input features. Hence, the physician may be left choosing between understandable explanations without formal guarantees and formally guaranteed explanations of restricted form.

This paper presents a substantially extended, integrated, and experimentally validated version of SpExplAIn, a tool for computing *and* interpreting space explanations of classifying deep neural networks with formal guarantees and domain-oriented visualizations. Relative to the earlier implementation [33], it supports ONNX models, including convolutional networks, reshape and flatten operations, and pooling, incorporates activation-phase fixing and guidance for deeper networks [34], and integrates geometric extraction and visualization as a reusable interpretation workflow [38]. These are implementation and integration contributions building on previously published algorithms, not claims that those algorithms are new. SpExplAIn is built around an explicit *separation of concerns* between the *computation* of an explanation and its *interpretation* [33, 34, 38]. The computation focuses solely on obtaining a *space explanation* [33,37]: a logical formula, correct by construction, that can capture relations between input features and, depending on the generalization strategy, may extend beyond the neighborhood of the sample. The interpretation then renders this formula, which may be too complex to be read directly, into a form tailored to the *end user*. In the example above, the physician computes the explanation once and then interactively explores it: SpExplAIn extracts a linear relation between age and cholesterol along which the classification provably remains unchanged and visualizes it in the space of these two features; the physician can then turn to, e.g., blood pressure and maximum heart rate without recomputing the explanation. For image recognition, SpExplAIn visualizes jointly allowed changes in pixel values while the classification is still guaranteed. Each extracted interpretation must itself preserve sufficiency for the classification. The end user does not need to understand the underlying methods: the computation runs with suitable defaults, and preferences such as "focus on these two features" are currently expressed through settings selected by the user, while NN-specific settings come from an NN expert or are automatic. Since the computation can be tuned to produce explanations with a desired property, such as *convexity*, the explanations are amenable to classical geometric analysis [38].

The main contributions of the extended tool are its features. It

- implements the computation of space explanations that are:
  - provably *correct* (i.e., sufficient conditions for the classification),
  - able to capture *relationships* between input features beyond feature-wise constraints (Sect. 4), and
  - potentially more general than explanations restricted to a neighborhood of the sample, as illustrated in Sect. 4;
- separates *computation* and *interpretation*: application-specific geometric extraction and visualization enable repeated and interactive use of one explanation (each extraction must preserve sufficiency);
- integrates scalable and *flexible* computation for deeper networks (Sect. 4): Craig interpolation derives explanations from a single proof-producing solver

check rather than feature-wise queries [13,37], complemented by architecture-guided computation [34]. Combinable generalization strategies, special cases of space explanations, and symbolic engines support method comparisons.

Related Work. Logic-based explainability of NNs [8, 25, 27, 33, 34, 37, 38, 43, 44, 55, 56, 60, 61] computes *sufficient conditions* for the classification of a sample through *abductive reasoning*, which originates from program verification [1, 16, 50, 51], using symbolic engines based on SMT [46, 47], such as SMT solvers [6, 10–12, 14, 18] or NN verifiers. Several prior methods iteratively relax constraints on individual input features without exploiting proofs of unsatisfiability, often requiring a series of expensive queries to a black-box solver. Their cost may depend on the number and order of features, and the resulting explanations are hyperrectangular, often restricted to a neighborhood of the sample. VeriX [61] and its successor VeriX+ [60] compute such explanations on top of the NN verifier Marabou [32] and visualize them as sets of relevant pixels; VeriX+ reduces the sensitivity to the number of features, but still relies on iterative queries, hence the remaining sensitivity is still a bottleneck. The approach of [8], implemented in Cora [3], accelerates the individual queries by abstracting and refining the network, while the sensitivity to the feature count remains. Interval explanations [27] widen the constraints on individual features, and distance-restricted explanations [26] are bounded by a distance from the sample, but neither can capture relations between features. Contrastive explanations [24, 28] describe which changes of the features would change the classification, dually to abductive ones [24]. NN verification tools, surveyed in [31, 35], check properties such as local robustness and serve as engines of the above tools, but do not by themselves provide interpolation-based explanation generalization. SpExplAIn builds on our prior work: [37] introduced space explanations computed by Craig interpolation, [34] scaled the computation to deep networks by fixing and guiding neuron activations, [38] proposed geometric interpretation techniques, and [33] described the broader framework; the present paper focuses on the extended implementation, integration, and evaluation of these methods.

Other approaches visualize individual neurons [19, 62], use gradients for local attributions [4, 57], or invert networks to approximate input representations [42]. Widely used model-agnostic tools [5] query the model on samples around the input: LIME [52] approximates it by a linear surrogate, SHAP [40] computes feature attributions rather than sufficient conditions, and Anchors [53] provides if-then rules whose precision is only estimated and which constrain each feature individually. These methods do not generally provide formal guarantees outside sampled points: their explanations may be logically inconsistent [44], may assign importance to irrelevant features [22], and can be fooled by models that behave differently on the perturbed samples [58].

2 Preliminaries

Neural Network Classifiers. A finite set of input *features* is $\mathcal{F} = \{1, \ldots, m\}$, each associated with a continuous *domain* $\mathcal{D}_i \subset \mathbb{R}$, a closed interval. A *classifier*

is $\mathcal{M} = (\mathcal{F}, \mathbb{F}, \mathcal{K}, \kappa)$, where $\mathbb{F} = \mathcal{D}_1 \times \mathcal{D}_2 \times \cdots \times \mathcal{D}_m$ is the input *feature space*, $\mathcal{K} = \{c_1, c_2, \ldots, c_K\}$ a finite set of K classes, $K \geq 2$, and $\kappa : \mathbb{F} \to \mathcal{K}$ is a *classification function*. A point $\mathbf{x} = (x_1, \ldots, x_m) \in \mathbb{F}$ consists of input *feature variables* $x_i \in \mathcal{D}_i$, and a *sample point* $\mathbf{s} = (s_1, \ldots, s_m) \in \mathbb{F}$ of constants. A *feedforward classifying neural network* (NN) is a classifier represented by a graph $(\mathcal{V}, \mathcal{E})$ of neurons and their weighted connections. Neurons $v_i^{(\ell)}$ are partitioned into $L+1$ layers $\mathcal{V}^{(0)}, \ldots, \mathcal{V}^{(L)}$ such that each $\mathcal{V}^{(\ell)} = \{v_i^{(\ell)}\}_{i=1}^{n^{(\ell)}}$ contains $n^{(\ell)}$ neurons. Layers $\mathcal{V}^{(0)}$ and $\mathcal{V}^{(L)}$ are the *input* and *output* layers, and the *hidden* layers in between are either *activation layers*, applying a nonlinear activation, or *linear layers*; $\mathcal{V}_{\mathcal{A}}$ denotes the set of *activation neurons*. An edge $(v_i^{(\ell)}, v_j^{(\ell+1)}) \in \mathcal{E}$ has a weight $w_{i,j}^{(\ell)} \in \mathbb{R}$, every non-input neuron $v_i^{(\ell)}$ has a bias $b_i^{(\ell)} \in \mathbb{R}$, and each input neuron $v_i^{(0)}$ and output neuron $v_i^{(L)}$ is associated with x_i and class c_i, respectively. The *pre-activation* $z_i^{(\ell)} : \mathbb{F} \to \mathbb{R}$ and *activation* $a_i^{(\ell)} : \mathbb{F} \to \mathbb{R}$ of $v_i^{(\ell)}$ are: $z_i^{(0)}(\mathbf{x}) = a_i^{(0)}(\mathbf{x}) = x_i$, and for hidden and output layers ($1 \leq \ell \leq L$):

$$z_i^{(\ell)}(\mathbf{x}) = \sum_{j=1}^{n^{(\ell-1)}} w_{j,i}^{(\ell-1)} a_j^{(\ell-1)}(\mathbf{x}) + b_i^{(\ell)} \tag{1}$$

$$a_i^{(\ell)}(\mathbf{x}) = \sigma\big(z_i^{(\ell)}(\mathbf{x})\big) \tag{2}$$

where $w_{j,i}^{(\ell-1)} = 0$ for absent edges, $\sigma(z) = ReLU(z) = \max\{z, 0\}$ in activation layers, and $\sigma(z) = z$ otherwise. An activation neuron v with pre-activation z_v is *active* at $\mathbf{x}$ if $z_v(\mathbf{x}) > 0$ and *inactive* otherwise, denoted by $\text{phase}_v(\mathbf{x})$. The classification is given by the output neuron with the maximum activation: $\kappa(\mathbf{x}) = c_i \iff i = \text{argmax}_j\{a_j^{(L)}\}$ with ties broken by an arbitrary fixed rule.

Space Explanations. *Space explanations* are logical explanations rooted in abductive reasoning: they state sufficient conditions for a classification.

Definition 1 (Space Explanation, Impact Space [34, 37]). *Given a classifier $\mathcal{M} = (\mathcal{F}, \mathbb{F}, \mathcal{K}, \kappa)$, and a class $c \in \mathcal{K}$, a* space explanation *of class c is a formula φ such that $\exists \mathbf{x} \in \mathbb{F}.\, \varphi(\mathbf{x})$ and $\forall \mathbf{x} \in \mathbb{F}.\, \varphi(\mathbf{x}) \implies \kappa(\mathbf{x}) = c$. The* impact space *of φ is the set $\mathbb{F}_\varphi := \{\mathbf{x} \in \mathbb{F} \mid \varphi(\mathbf{x})\} \subset \mathbb{F}$.*

For a sample point $\mathbf{s} \in \mathbb{F}$, the *sample explanation* $\varphi_{\mathbf{s}}(\mathbf{x}) := \bigwedge_{i \in \mathcal{F}} x_i = s_i$ has impact space $\mathbb{F}_{\varphi_{\mathbf{s}}} = \{\mathbf{s}\}$. Space explanations guarantee the classification on their whole impact space and are not restricted in shape or structure, hence can capture relations between input features. Unlike many prior concepts, they do not require a designated sample point; to explain the prediction for a sample $\mathbf{s}$, we additionally require $\varphi(\mathbf{s})$, which our computation ensures by generalizing $\varphi_{\mathbf{s}}$.

Explanation Computation. Given a classifier $\mathcal{M}$, a class $c \in \mathcal{K}$, and a sample $\mathbf{s}$ with $\kappa(\mathbf{s}) = c$, computing a space explanation φ amounts to *generalizing* $\varphi_{\mathbf{s}}$ so that $\varphi_{\mathbf{s}} \Rightarrow \varphi$ and $\varphi(\mathbf{x}) \Rightarrow \kappa(\mathbf{x}) = c$ on $\mathbb{F}$. With proof-based interpolation, the key steps are:

1. Check that $\varphi_{\mathbf{s}}(\mathbf{x}) \wedge \kappa(\mathbf{x}) \neq c$ is unsatisfiable and obtain a proof.
2. Use the proof to generalize $\varphi_{\mathbf{s}}$ into φ while keeping $\varphi(\mathbf{x}) \wedge \kappa(\mathbf{x}) \neq c$ unsatisfiable.

Thus, one proof-producing solver call suffices for interpolation-based generalization, regardless of the number of input features (though solving time may still depend on that number). A previously computed space explanation of the same class can serve as the starting point instead of $\varphi_{\mathbf{s}}$.

Classification Query Encoding. Using an SMT-LIB [7] quantifier-free first-order logic, $\kappa(\mathbf{x}) \neq c$ is encoded by $\psi := \psi_{\mathcal{M}} \wedge \psi_{\mathcal{D}} \wedge \neg\psi_c$:

$\psi_{\mathcal{M}}$: The network $\mathcal{M}$: constraints (1) and (2) with fixed weights and biases, $ReLU$ encoded by if-then-else (ITE) terms.

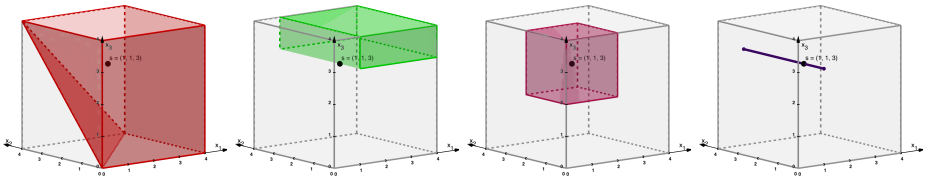
$\psi_{\mathcal{D}}$: The input domains $x_i \in \mathcal{D}_i$ for all $i \in \mathcal{F}$.

$\neg\psi_c$: The classification is *not* c: at least one other output neuron $a_i^{(L)}(\mathbf{x})$ *reaches* the value of class c; treating ties as a change of the classification makes the explanations correct regardless of the tie-breaking rule.

Functions $z_i^{(\ell)}$ and $a_i^{(\ell)}$ are inlined or encoded by auxiliary variables. This encoding is general; the presented tool instantiates the logic to QF_LRA (Quantifier-Free Linear Real Arithmetic). Hence, the explanations are correct w.r.t. the exact real-arithmetic semantics of the network, with weights and biases as exact rationals, rather than its floating-point implementation, which may differ in rounding close to decision boundaries. Floating-point encodings (e.g., QF_FP) are outside the present scope.

Generalization Techniques. Given an unsatisfiable formula $A \wedge B$, a *Craig interpolant* [13] is a formula I s.t. $A \Rightarrow I$, $I \wedge B$ is unsatisfiable, and I uses only the common variables of A and B. $I = Itp(A, B)$ denotes the interpolant computed by an interpolation algorithm Itp from a proof of the unsatisfiability of $A \wedge B$. For a space explanation φ (e.g., $\varphi_{\mathbf{s}}$), $I = Itp(\varphi, \psi)$ is again a space explanation, not restricted in shape, s.t. $\varphi \Rightarrow I$ (hence $\mathbb{F}_{\varphi} \subseteq \mathbb{F}_I$). Interpolation algorithms yield formulas of different strengths and structures [17, 54]. The propositional McMillan algorithm (Itp_M) typically yields conjunctions, while its dual (Itp'_M) favors disjunctions [45]. For QF_LRA, the Farkas-based methods (Itp_F, Itp_{DF}) and their duals (Itp'_F, Itp'_{DF}) [9] offer different strengths: Itp_{DF} and Itp_F yield stronger interpolants than their duals, while the parameterized method (Itp_f, $f \in [0, 1]$) [2] provides an intermediate choice on the same inputs. Combining McMillan with Farkas-based interpolation, excluding Itp'_{DF}, yields conjunctions of linear inequalities, hence convex impact spaces suitable for geometric extraction; convexity follows from this conjunctive form, and disjunctive results need not be convex. Partitioned interpolation [21] generalizes selected constraints of an explanation while retaining the others, allowing users to focus on particular features without losing the classification guarantee.

For a conjunctive explanation φ with $\varphi \wedge \psi$ unsatisfiable, an *unsatisfiable core* is a subset of its conjuncts that remains inconsistent with ψ. The omitted conjuncts can be dropped, yielding a weaker explanation; the core is *irreducible*



(a) Space explanation **(b)** Interval explanation **(c)** [8, 26, 61] **(d)** [25]

Fig. 1: Visual comparison of space explanations and their special cases in 3D.

if no further conjunct can be dropped. Core extraction can also target selected constraints; for $\varphi_{\mathbf{s}}$, omitting equalities frees the corresponding features [25].

Explanation Interpretation. A space explanation φ can be difficult to interpret directly. An *extraction* selects from φ a simpler, satisfiable formula $\varphi^{\downarrow}$ with $\varphi^{\downarrow} \Rightarrow \varphi$, hence again a correct space explanation [38]. A *slice* keeps features $\mathcal{X} \subseteq \mathcal{F}$ free and fixes all others to the sample: $\varphi^{\downarrow} := \varphi \wedge \bigwedge_{i \in \mathcal{F} \setminus \mathcal{X}} x_i = s_i$. It can be visualized in two or three dimensions, but only holds for the fixed values. For a closed, convex impact space $\mathbb{F}_{\varphi}$, a *maximum diameter* $\operatorname{argmax}_{\mathbf{x}^1, \mathbf{x}^2 \in \mathbb{F}_{\varphi}} \|\mathbf{x}^1 - \mathbf{x}^2\|_2^2$, exposes a direction along which features can vary together. By convexity, for endpoints $p, q \in \mathbb{F}_{\varphi}$, the segment $\varphi^{\downarrow}_{p,q}(\mathbf{x}) := \exists t \in [0,1]\,.\ \mathbf{x} = p + t(q - p)$ is a space explanation constraining all coordinates. The selected interpolation output must satisfy the stated convexity and closedness assumptions, including when strict inequalities occur. For normalized domains $\mathcal{D}_i = [0,1]$, the *minimum norm point* minimizes $\|\mathbf{x}\|_2$ over $\mathbb{F}_{\varphi}$, whereas the *inverted minimum norm point* minimizes $\|\mathbf{1} - \mathbf{x}\|_2$, i.e., is closest to the upper domain corner; both are representative points [38, 48].

Special Cases. Earlier abduction-based explanation concepts [8, 25, 27, 43, 44, 55, 56, 60, 61] can be viewed as special cases of space explanations [33]. Many use a fixed, predefined form, such as individual feature intervals or a norm ball around the sample [26], which restricts the relationships they can express. Individual-feature constraints in particular are encompassed by interval explanations.

Definition 2 (Interval Explanation [27, 33]). *A space explanation $\varphi_{\mathcal{I}}$ is an* interval explanation *if $\varphi_{\mathcal{I}} \equiv \bigwedge_{i \in \mathcal{X}} l_i \leq x_i \leq u_i$ for some $\mathcal{X} \subseteq \mathcal{F}$ and $[l_i, u_i] \subseteq \mathcal{D}_i$.*

Its impact space is the hyperrectangle $\mathbb{F}_{\varphi_{\mathcal{I}}} = \prod_{i \in \mathcal{F}} [l_i, u_i]$ with $[l_i, u_i] = \mathcal{D}_i$ for $i \notin \mathcal{X}$. Fig. 1 compares these forms for $\mathbb{F} = [0,4]^3$ and $\mathbf{s} = (1,1,3)$. The space explanation (a) covers a larger region and captures a relation among all three features. The displayed interval explanation (b) and sample-anchored forms (c)(d) do not express this relation.

3 SpExplAIn Framework

Fig. 2 outlines the two components of SpExplAIn, separated by a common space-explanation representation φ: the **Explanation Computation Engine** establishes a sound and potentially general explanation, while the **Explanation**

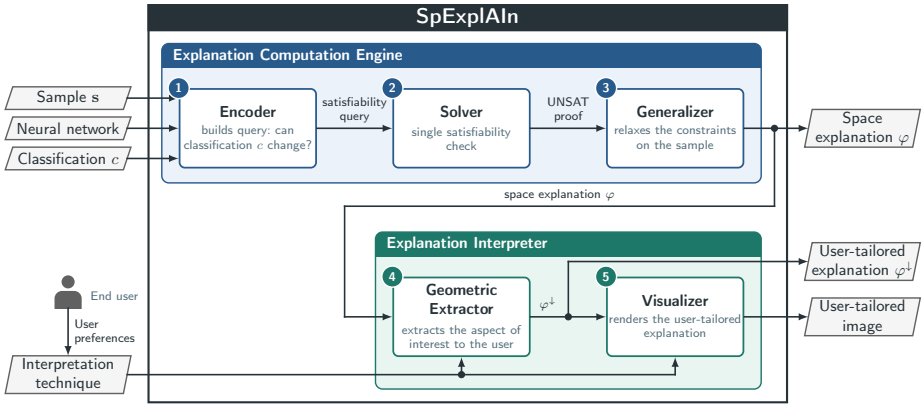


Fig. 2: High-level overview of SpExplAIn: the Explanation Computation Engine (steps 1–3) and the Explanation Interpreter (steps 4–5).

Interpreter presents selected aspects in a *human-oriented* form. The components are logically decoupled: the engine need not restrict explanations to an easily visualized form, and the interpreter need not know how φ was computed. The sample and feature metadata may accompany φ during interpretation.

For a neural network, a sample $\mathbf{s}$, and its classification c, the **Encoder** (step 1) builds a query for a possible change of class. The **Solver** (step 2) checks this query together with the sample, and the **Generalizer** (step 3) relaxes the sample constraints. With interpolation, one proof-producing UNSAT check suffices for this generalization; other strategies may require further solver checks. The resulting explanation can express *relations* between features and extend beyond the sample's neighborhood without a predefined shape. The engine supports different generalization strategies and symbolic backends, including configurations for deeper networks guided by activation phases (Sect. 4).

The **End user**, a domain expert rather than a formal-methods expert, chooses which aspect of φ to inspect. The **Geometric Extractor** (step 4) obtains a user-tailored explanation $\varphi^{\downarrow}$ that preserves sufficiency, and the **Visualizer** (step 5) renders it. Different views can be produced from the same φ without repeating the computation.

We now describe the computation, interpretation, and usage of the tool.

Computing Space Explanations. Fig. 3 expands the computation steps into interfaces for encoding, solving, and generalization. The **NN Encoder** builds the classification-change query ψ (Sect. 2), independently of the sample when no phase-based configuration is applied. Together with $\varphi_{\mathbf{s}}$, it forms the unsatisfiable input $\psi \wedge \varphi_{\mathbf{s}}$ for a successful computation. For the `itp` strategy, the **Solver** obtains an UNSAT proof, from which the **Generalizer** derives an intermediate explanation φ'. Other strategies use the same solver interface, possibly with multiple checks. Without activation fixing ($\mathcal{F}_{\mathcal{A}} = \emptyset$), the final explanation is $\varphi = \varphi'$. Instead of $\varphi_{\mathbf{s}}$, a previously computed, user-tailored, or other sound space explanation can serve as the starting formula [37].

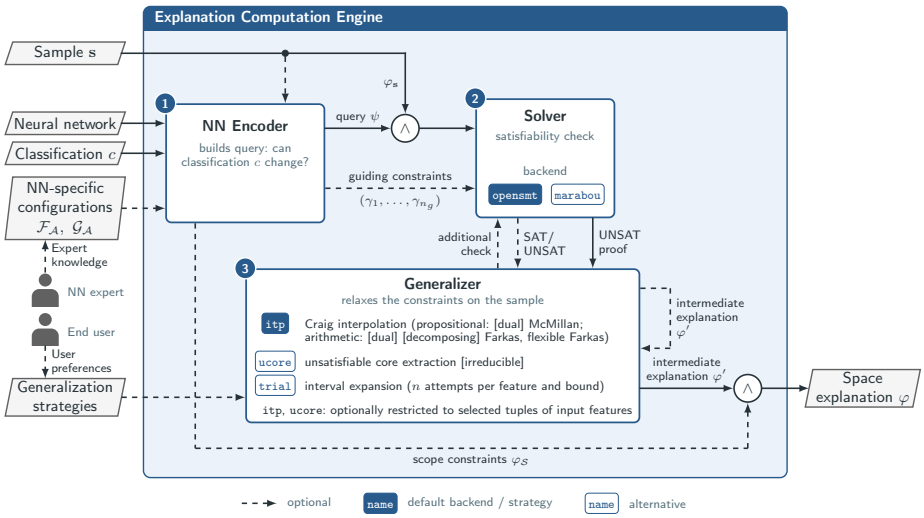


Fig. 3: SpExplAIn Explanation Computation Engine.

For an unrestricted encoding, successful generalization from a starting explanation φ_0 satisfies

$$\varphi_0 \Rightarrow \varphi, \qquad \varphi \wedge \psi \text{ is unsatisfiable.}$$

Thus φ preserves classification c; when $\varphi_0 = \varphi_\mathbf{s}$, it also contains $\mathbf{s}$. With activation fixing, the corresponding conditions first apply to φ' under the phase-restricted encoding. Conjoining the scope constraints $\varphi_\mathcal{S}$ then restores the classification guarantee for the original network and, when starting from $\varphi_\mathbf{s}$, retains the sample, as detailed below. These guarantees assume a faithful network encoding and sound solver results.

End User and NN Expert. The end user's goals are expressed as *user preferences* such as "make the explanation as general as possible", "make it suitable for a geometric interpretation", "focus on the relation of these two features", or "give me just the bounds of individual features", which translate to particular *generalization strategies* and their parameters (see the **Generalizer** below). SpExplAIn does not perform this translation automatically yet; instead, the end user follows the manual at the end of this section. The default strategy is suitable for most of the use cases. In contrast, the *NN-specific configurations* ($\mathcal{F}_\mathcal{A}$ and $\mathcal{G}_\mathcal{A}$) represent *expert knowledge* provided by an **NN expert** or computed automatically, so the end user can rely on the defaults.

NN Encoder. The **NN Encoder** reads the network either in the `.nnet` format for fully connected networks or in the general `.onnx` format, where it supports fully connected and convolutional layers (hence CNNs), flatten and reshape operations, and average, minimum, and maximum pooling. Linear operations are encoded as the pre-activations (1) over the actual edges of the network graph. Only *ReLU* activations are currently implemented (but the generic encoding can be extended, as clarified in Sect. 2); they are represented as explicit layers that

turn the preceding layer into an activation layer and, like minimum and maximum pooling, are encoded using ITE terms. Following the algorithms of [34], adapted to explicit activation layers, the encoding is parametrized by sets of activation neurons whose activation phase at $\mathbf{s}$ is *fixed* ($\mathcal{F}_\mathcal{A}$) or *guided* ($\mathcal{G}_\mathcal{A}$), which prunes or prioritizes the *ReLU* case splits and substantially speeds up the solving for deep networks. For each neuron in $\mathcal{F}_\mathcal{A}$, only the *ReLU* branch active at $\mathbf{s}$ is encoded, and the corresponding activation condition is added to *scope constraints* $\varphi_\mathcal{S}$. More precisely, for each activation neuron $v \in \mathcal{V}_\mathcal{A}$ we define

$$S_v(\mathbf{x}) = \begin{cases} z_v(\mathbf{x}) > 0, & z_v(\mathbf{s}) > 0, \\ z_v(\mathbf{x}) \leq 0, & z_v(\mathbf{s}) \leq 0, \end{cases} \qquad \text{and} \quad \varphi_\mathcal{S}(\mathbf{x}) = \bigwedge_{\substack{v \in \mathcal{F}_\mathcal{A} \\ |z_v(\mathbf{s})| > \varepsilon}} S_v(\mathbf{x}).$$

The stability condition reflects that neurons with $|z_v(\mathbf{s})| \leq \varepsilon$ are never fixed. The resulting explanation $\varphi := \varphi_\mathcal{S} \wedge \varphi'$ (φ' is the output of the **Generalizer**) is then sound w.r.t. the original network [34]. For each neuron in $\mathcal{G}_\mathcal{A}$, the **NN Encoder** produces a *guiding constraint* γ_i that suggests the activation phase at $\mathbf{s}$ to the **Solver**; guiding constraints affect neither soundness nor correctness. Both sets can be provided by the **NN expert** or computed automatically by external NN analysis algorithms. Besides guiding all neurons ($\mathcal{G}_\mathcal{A} = \mathcal{V}_\mathcal{A}$), the preferred default for $\mathcal{G}_\mathcal{A}$ that improves both performance and generality in practice [34], and fixing all neurons ($\mathcal{F}_\mathcal{A} = \mathcal{V}_\mathcal{A}$), an *adversarial example* for $\mathbf{s}$ is a point $\mathbf{s}' \in \mathbb{F}$ with $\kappa(\mathbf{s}') \neq \kappa(\mathbf{s})$. For such a point, the phase-changing activation neurons are

$$\mathcal{R}_\mathcal{A}(\mathbf{s}, \mathbf{s}') := \{v \in \mathcal{V}_\mathcal{A} \mid \text{phase}_v(\mathbf{s}) \neq \text{phase}_v(\mathbf{s}')\}. \tag{3}$$

SpExplAin can also automatically compute a hybrid configuration [34] that fixes neurons in $\mathcal{V}_\mathcal{A} \setminus \mathcal{R}_\mathcal{A}$ and guides the remaining neurons; $\mathbf{s}'$ can be computed using Projected Gradient Descent [41]. Without fixing, $\varphi_\mathcal{S} = \top$ and $\varphi = \varphi'$. The NN configurations for $\mathcal{G}_\mathcal{A}$ and $\mathcal{F}_\mathcal{A}$ substantially improve the performance over the unguided encoding, as illustrated in Sect. 4.

Solver. The default backend of the **Solver** is `opensmt` [10, 23], an interpolating SMT solver efficient in QF_LRA that natively supports (irreducible) unsatisfiable core extraction and decision guidance [34]. The alternative backend `marabou` [32] is often faster at NN-specific satisfiability checks but supports neither interpolation, unsatisfiable cores, nor guidance, and hence only computes interval explanations. More backends can be added through the abstraction layer.

Generalizer. Generalization strategies can be chained [37], with each strategy receiving the preceding result. The default `itp` strategy (Craig interpolation) uses the UNSAT proof to derive feature relations. Selecting among the interpolation algorithms, and combining the propositional and arithmetic ones, yields *variations* of explanations with different logical strengths and structures: for instance, the preference for an explanation as general as possible translates to a logically weak algorithm (e.g., dual Farkas), while geometric interpretation calls for the conjunctive output of a suitable Farkas-based arithmetic and propositional interpolation combination: a conjunction of linear constraints defines a convex impact space. The preference to focus on, e.g., a pair of features translates to restricting

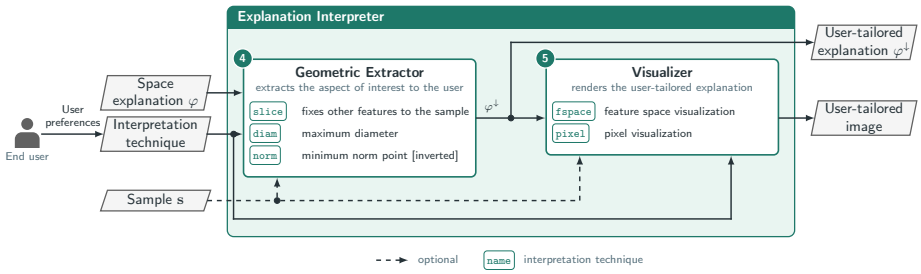


Fig. 4: SpExplAIn Explanation Interpreter.

the generalization to selected tuples of input features by partitioning the input, which also speeds up the interpolation. The strategy `ucore` (unsatisfiable core extraction, optionally irreducible) reduces a conjunctive explanation, either the input (e.g., $\varphi_{\mathbf{s}}$) or an intermediate result, and can be restricted to selected tuples of input features as well; applied to $\varphi_{\mathbf{s}}$, it identifies the input features that are irrelevant for the classification. The strategy `trial` (interval expansion [33, 37], a modification of [27]) widens the bounds of individual features by a bounded binary search with n attempts per feature and bound, which corresponds to the preference for bounds of individual features; here, `marabou` is often the best backend. Unlike `itp`, `trial` and the irreducible `ucore` require additional checks by the **Solver**, are sensitive to the number and order of conjuncts in the input, and do not exploit the proof. New strategies can be added by implementing the interface for the explanation types they support.

Interpreting Space Explanations. Fig. 4 details the Explanation Interpreter, which renders a computed space explanation φ into representations intended for domain-specific interpretation by the **End user**. Unlike computation, which typically uses defaults, interpretation starts with a domain-specific question, such as "how do age and cholesterol relate?" The end user then selects an *interpretation technique*, consisting of an extraction and a visualization technique including their parameters such as the features of interest. The **Geometric Extractor** produces the *user-tailored explanation* $\varphi^{\downarrow}$ (Sect. 2); the **Visualizer** produces its image. Optionally, both components take the sample $\mathbf{s}$ to anchor the result at the explained point. Both $\varphi^{\downarrow}$ and the image are returned: what the end user sees remains a space explanation that guarantees the classification, in contrast to, e.g., saliency maps [57]. Moreover, $\varphi^{\downarrow}$ can be further reused by SpExplAIn, e.g., as the starting point of the computation. The guarantee concerns the network only: whether an extracted relation is meaningful in the application domain, e.g., medically, remains the judgment of the end user.

Geometric Extractor. The `slice` technique fixes all features except the selected ones to the sample [37], typically a pair or a triplet of features that the user considers related, such as age and cholesterol for a physician or income and loan amount for a banker. The `diam` technique extracts an estimated maximum diameter of the impact space, estimated by maximizing the

distance of random pairs of points on its surface [38]. The estimation works in any number of dimensions: the end user can apply it to the whole impact space, or to a slice of the selected features. The result is a short formula, e.g., $(0.3 \leq x_1 \leq 1) \wedge (x_5 = -0.096x_1 + 0.24) \wedge \bigwedge_{i \in \mathcal{F} \setminus \{1,5\}} x_i = s_i$ (with coefficients rounded for presentation) for the slice of the age x_1 and cholesterol x_5 of a heart-attack risk prediction [29, 38], which tells the user how the features may change together without changing the classification, provided the other features satisfy the constraints of the full extracted segment. The variance of the diameter lengths indicates whether such a relation is significant [38]. The `norm` technique extracts the minimum or the inverted minimum norm point, which is suitable for image recognition: with a dark background, the minimum norm point is a representative image, valid as a whole, that is as close to black as the explanation allows, illustrating how far the pixels can be darkened *together* while the classification is still guaranteed [34, 38]; with a light background (inverted colors), the inverted minimum norm point analogously shows how far the pixels can be brightened together. Both `diam` and `norm` require a closed convex impact space; if not closed, the results are approximate and may lie on its boundary.

Visualizer. The `fspace` technique renders the impact space of $\varphi^{\downarrow}$ within the two- or three-dimensional space of the selected features, together with the sample point, which suits tabular data where the user reasons about concrete features. The `pixel` technique renders a point of $\varphi^{\downarrow}$, such as the minimum norm point, as an image of the same shape as the input, which suits image recognition where the user is interested in the whole image rather than in individual pixels.

Interactive Use. While the computation of φ may be expensive for deep networks, the interpretation takes only seconds, even for explanations occupying megabytes [34, 38]. Hence, the interpreter is typically run interactively and repeatedly for the same explanation and sample, each time focusing on a different aspect. For example, using `diam`, a physician first inspects the relation between age and cholesterol in the explanation, then between blood pressure and maximum heart rate, without recomputing φ.

SpExplAIn Usage. SpExplAIn is publicly available[4], where the README files describe installation, options, and examples. It is implemented in C++ and Python, with auxiliary scripts in Bash, and run as `spexplain <action>`. The default action `explain` runs the Explanation Computation Engine on `<model> <dataset> [<strategies>]`: a NN in the `.nnet` or `.onnx` format, a CSV file of samples to be explained, and a sequence of chained generalization strategies and their parameters, e.g., `'itp aweak, bstrong; ucore'`. The preferences of the end user translate to strategies as follows. The default `itp aweak, bstrong`, i.e., Itp'_F (dual Farkas) with Itp_M (McMillan), yields general explanations with a convex impact space, suitable for all interpretation techniques; `aweaker` yields even more general, but non-convex explanations. The parameter `vars x<i>...`

[4] https://github.com/usi-verification-and-security/spexplain

focuses `itp` or `ucore` on selected features, `min` makes `ucore` irreducible and applied to the samples identifies irrelevant features; `trial` computes interval explanations, preferably with `--verifier marabou` (must be explicitly built: `make marabou`). Previously computed or user-tailored explanations can be further generalized via `--input-explanations`. The NN expert provides the NN-specific configurations as files of neuron activations via `--input-fix-sample-neuron-activations` and `--input-guide-sample-neuron-activations`, or selects fixing all neurons or the hybrid configuration (3) via `--fix-default-sample-neuron-activations` (`all` or `adversarial`); guiding defaults to `all`. The action `interpret` runs the Explanation Interpreter on `<explanation> <technique> [<dataset>]`: a stored explanation and an extraction technique, e.g., `slice x1 x5`, `diam`, or `norm` (optionally `inv`), which determines the visualization (`fspace`, currently in 2D, or `pixel`) triggered by `--visualize`; a visualization technique can also be applied directly. Since the explanation is stored, the interpretation can be repeated with different techniques. Finally, scripts in `data/scripts` run batches of experiments and analyze the explanations, e.g., by subset checks or by `check`, which re-verifies them with an external SMT solver in QF_LRA.

4 Experimental Evaluation

We evaluate how SpExplAIn delivers the generality of the computed explanations and their ability to capture relations between input features, their interpretation by the Explanation Interpreter, and the scalability and flexibility of the computation, and compare it with the state of the art. We use three tabular datasets, heart attack risk [29] (13 inputs, 2 classes), obesity [49] (15 inputs, 7 classes), and loan [39] (22 inputs, 2 classes), and three image classification datasets, GTSRB [59] ($32 \times 32 \times 3$ inputs, 43 classes), CIFAR10 [36] ($32 \times 32 \times 3$ inputs, 10 classes), and MNIST [15] (28×28 inputs, 10 classes). Tabular data are classified by fully connected (FC) networks, and image data by both FC and convolutional networks (CNNs). The FC networks have 1 to 14 hidden layers with 50 or 200 neurons each. We use 24 CNN architectures (Table 1 in Appendix A): six depths $d \in \{1, \ldots, 6\}$ (number of convolutional layers) times four variants given by two choices. (i) *Downsampling*: after the first, strided convolution, the networks downsample by stride-2 convolutions (Sd) or by $2{\times}2$ average pooling (ABd). (ii) *Width*: wide (8/16/32 channels) or narrow (prefix N, 4/8/16 channels). Since *ReLU* is applied before pooling, ABd has exactly as many *ReLU* splits as Sd. Batch normalization is used in training and folded into the preceding convolution at export.

Architecture details and training procedures are provided in Appendix A. We compare four configurations of the NN Encoder (Sect. 3): *Direct* ($\mathcal{G}_\mathcal{A} = \mathcal{F}_\mathcal{A} = \emptyset$, as in [37]), *Guide-all* ($\mathcal{G}_\mathcal{A} = \mathcal{V}_\mathcal{A}$, $\mathcal{F}_\mathcal{A} = \emptyset$, as in [34]), *Fix-all* ($\mathcal{G}_\mathcal{A} = \mathcal{V}_\mathcal{A}$, $\mathcal{F}_\mathcal{A} = \mathcal{V}_\mathcal{A}$), and *Hybrid* ($\mathcal{G}_\mathcal{A} = \mathcal{V}_\mathcal{A}$, $\mathcal{F}_\mathcal{A} = \mathcal{V}_\mathcal{A} \setminus \mathcal{R}_\mathcal{A}$ by (3) with an adversarial example $\mathbf{s}'$); unstable neurons are never fixed, and fixed neurons are not guided. The interpolation algorithms (Sect. 2) are denoted by $Itp_{DF} \mapsto$ `stronger`, $Itp_F \mapsto$ `strong`, $Itp_f \mapsto$ `mid` with $f := 0.5$, $Itp'_F \mapsto$ `weak`, and $Itp'_{DF} \mapsto$ `weaker`.

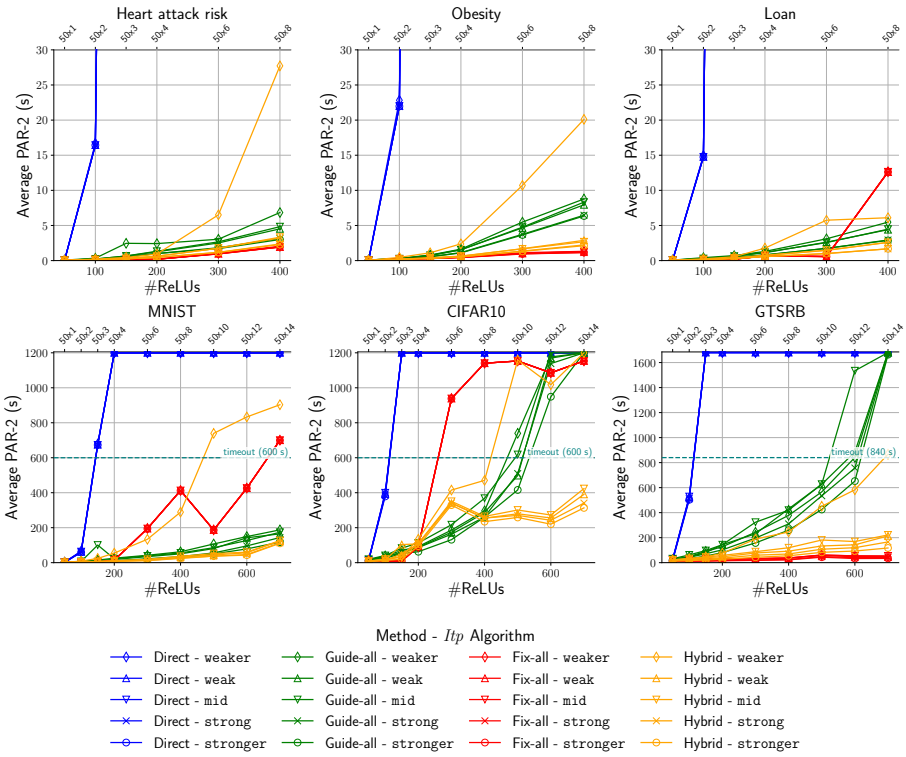


Fig. 5: PAR-2 runtime (s) of all configurations and interpolation algorithms w.r.t. the depth of the FC networks with 50 neurons per layer.

Unless stated otherwise, we use the default `weak`, which yields convex explanations suitable for all interpretation techniques. The time limit per explanation is 14 min for GTSRB, 10 min for MNIST and CIFAR10, and 5 min for tabular datasets. To aggregate runtimes and timeouts, we use the PAR-2 metric common in the SMT community: the runtime, or twice the time limit in the case of a timeout.

Fig. 5 shows the PAR-2 runtime of each configuration combined with each interpolation algorithm for FC networks with 50 neurons per hidden layer, with up to 8 hidden layers for the tabular datasets and up to 14 for the image datasets. The guided configurations substantially improve over *Direct*, which times out already on networks with three hidden layers. Importantly, all five interpolation algorithms scale similarly, so the end user can choose the logical strength of the explanations, from the strongest (`stronger`) to the weakest (`weaker`) [37], according to the preferences rather than to the performance.

Fig. 6 compares explanations of selected heart attack and loan samples on two-dimensional slices (`slice` and `fspace`). We omit *Hybrid*, whose explanations are not convex, and compare with the state-of-the-art tool VeriX+, which computes interval explanations within a perturbation radius ϵ around the sample; we

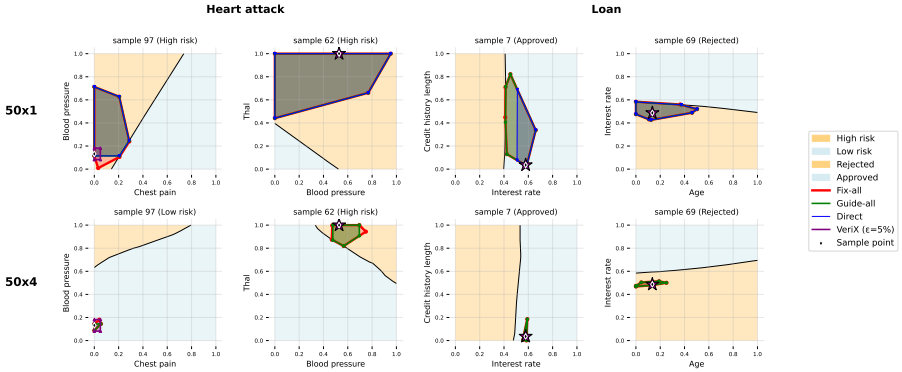


Fig. 6: Slices of explanations of SpExplAIn configurations (`weak`) and VeriX+ ($\epsilon = 5\%$) for heart attack and loan samples on the 50×1 (top) and 50×4 (bottom) networks.

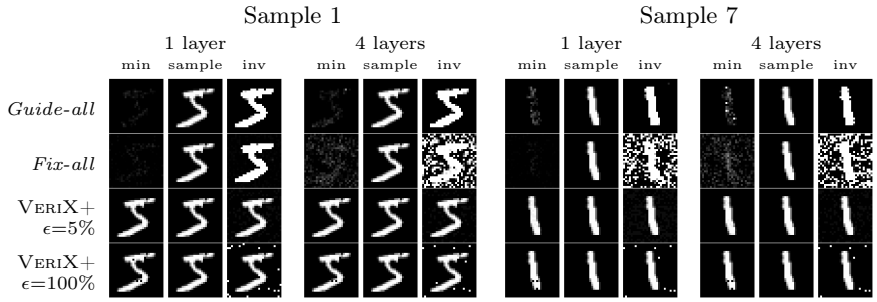


Fig. 7: Minimum norm point (min), sample, and inverted minimum norm point (inv) of the explanations of two MNIST samples computed by *Guide-all*, *Fix-all*, and VeriX+ on FC networks with 1 and 4 hidden layers of 50 neurons.

adopt $\epsilon \in \{0.5\%, 5\%, 100\%\}$ from [61] and show $\epsilon = 5\%$, since $\epsilon = 100\%$ returned only the sample itself on these networks. On the one-layer networks, our explanations cover large regions not restricted in shape that extend along the decision boundary, with their longer edges roughly parallel to it, and capture relations between the selected features, while VeriX+ is confined to a small box or the sample itself, contained in our explanations in most slices. On the four-layer networks, all explanations are considerably smaller (although they often still follow the decision boundary), presumably because these networks are unnecessarily complex for the simple tabular tasks with only 13 to 22 inputs: their decision regions are composed of many more linear pieces, which limits the explanations derived from them. Although *Fix-all* explanations are restricted by the scope constraints $\varphi_{\mathcal{S}}$, they are usually at least as general as those of the other configurations. For MNIST, Fig. 7 uses `norm` and `pixel`: for each sample and depth, the minimum norm point, the sample, and the inverted minimum norm point form a continuous series of images between them with the same guaranteed classification. *Fix-all* yields the most general explanations, whereas VeriX+ changes only a few isolated pixels for $\epsilon = 100\%$, and the differences from the sample are

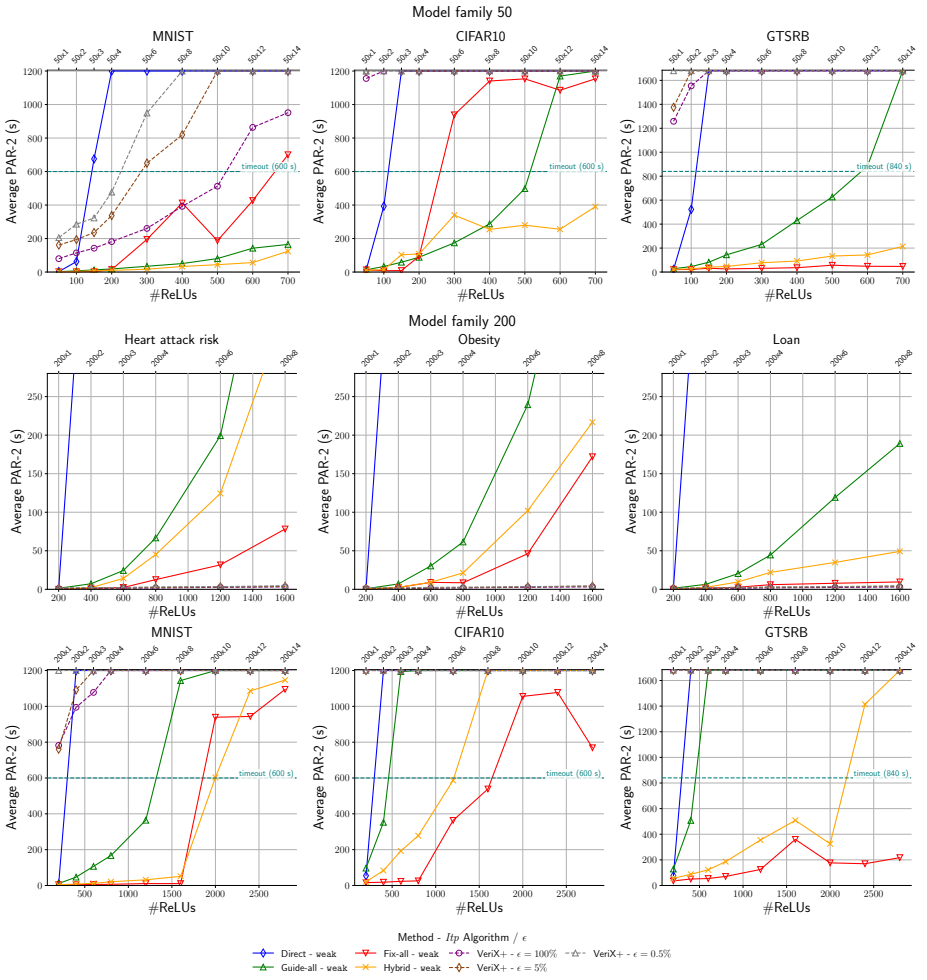


Fig. 8: PAR-2 runtime (s) of SpExplAIn (`weak`) and VeriX+ ($\epsilon \in \{0.5\%, 5\%, 100\%\}$) w.r.t. the depth of the FC networks with 50 (top) and 200 (bottom) neurons per layer.

barely visible for $\epsilon = 5\%$. In contrast to the tabular datasets, the explanations remain similar for networks with one and four layers, since the deeper network is adequate for the more complex task.

Fig. 8 shows the PAR-2 runtime of our configurations (400s aweak) and of VeriX+ on FC networks with 50 and 200 neurons per hidden layer; refer to Fig. 12 in Appendix B for tabular data with 50 neurons where VeriX+ performs similarly to SpExplAIn. No configuration is best everywhere: *Fix-all* is usually fastest on the tabular datasets and on GTSRB, *Hybrid* on CIFAR10 and MNIST with 50 neurons per layer, where it explains even the deepest networks within the time limit; the deeper image networks with 200 neurons per layer remain challenging. On the tabular datasets with only 13 to 22 inputs, VeriX+ is competitive on the narrower networks and faster than SpExplAIn on the wider

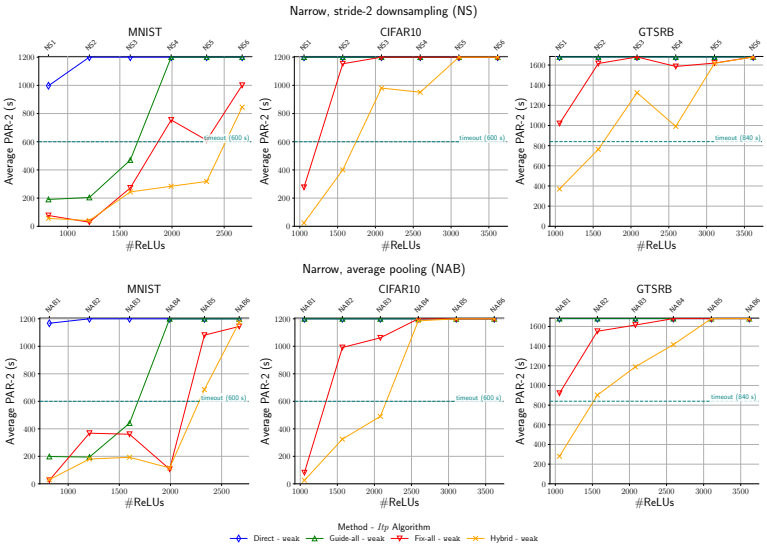


Fig. 9: PAR-2 runtime (s) of SpExplAIn w.r.t. #*ReLU*s and depth of narrow CNNs.

ones. On the image datasets, however, VeriX+ times out on almost all networks with 200 neurons per layer; with 50 neurons per layer, it times out on CIFAR10 and GTSRB and is slower than *Guide-all* and *Hybrid* on MNIST. This stems mainly from the sensitivity of VeriX+ to the number of input features, while SpExplAIn computes each explanation by a single solver call. We could not run the explanation procedure of Cora [8], for which no external comparison is available to the best of our knowledge; since it traverses the input features one by one, without divide-and-conquer techniques [30], we expect it to be even more sensitive to their number than VeriX+. Fig. 9 shows the results for the narrow CNNs (wide CNNs are shown in Appendix B). Despite considerable effort, we could reproduce the results reported for VeriX+ [60] only on FC networks; on our CNNs, it did not produce meaningful results, so we exclude it from the CNN evaluation. The CNNs have substantially more *ReLU* neurons and wider layers than the FC networks, hence much larger linear systems. *Hybrid* still explains the MNIST CNNs with up to about 2,300 *ReLU* neurons and the smaller CIFAR10 and GTSRB CNNs within the time limit, while the largest CNNs mostly time out.

SpExplAIn can further reduce an interpolant to an (optionally irreducible) unsatisfiable core (`itp; ucore [min]`), which yields a smaller formula with a larger impact space. The benefit depends on the configuration (Fig. 10). For *Direct*, the reduction simplifies the explanations substantially, e.g., from 48 to 29 (`ucore`) and 10 terms (`ucore min`) for sample 54 on the 50×1 network, but at a high cost: the average runtime grows from 0.05 s to 0.9 s and 225 s, respectively, and `ucore min` explains only 54 of 100 samples within 20 min. For *Fix-all*, the overhead is much lower, but the interpolant is already nearly irreducible: the cores coincide with it in 96–99% (`ucore`) and 82–95% (`ucore`

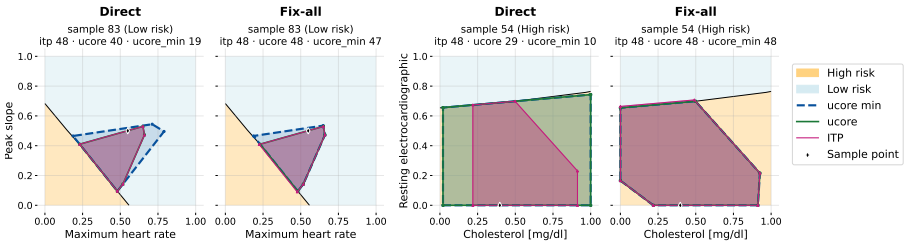


Fig. 10: `weak` explanations of *Direct* (top) and *Fix-all* (bottom) on the heart attack 50×1 network, reduced by `ucore` and `ucore min`; plot titles give the number of terms.

`min`) of the samples on the 50×1 and 50×4 networks. With the choice of interpolation algorithm (Fig. 5), strategies can thus be combined to trade off generality, size, and runtime.

All visual comparisons in this section are produced by the Explanation Interpreter (Sect. 3) from explanations computed once and stored, showing how the same formulas serve different domains. For tabular data, `slice` with `fspace` (Figs. 6 and 10) shows how a selected pair of features may change together without changing the classification, while the remaining features are fixed to the sample; comparing the polygon with the region of the sample's class shows how much of this region the explanation covers. For images, `norm` with `pixel` (Fig. 7) shows how far all pixels can be darkened or brightened together while the classification is still guaranteed. Since a more general explanation yields a slice at least as large and a minimum norm point of at most the same norm, these interpretations also compare the generality of explanations at a glance. Each interpretation is again a space explanation, and switching between techniques or features requires no recomputation. This enables the interactive workflow from the introduction: the physician explores one stored explanation of a patient step by step, e.g., through the relations in the first row of Fig. 11. Finally, `diam` condenses an explanation into a single linear relation along which the classification provably remains unchanged. Fig. 11 shows the extracted segments in the original units of selected pairs of features, with the remaining features fixed to the sample. In the first row, for a heart attack patient assessed as low-risk, `diam` tells the physician that the network keeps the low-risk assessment for patients aged between 63 and 71 years whose cholesterol rises with age from about 120 to 480 mg/dl, roughly 45 mg/dl per year of age, while the other characteristics stay as for this patient. Similarly, the cholesterol may range from about 130 to 460 mg/dl while the maximum heart rate rises only from about 156 to 180 bpm, and the assessment still holds. The second row shows analogous relations for a loan approval sample, between financial attributes such as the loan amount, credit history length, credit score, and interest rate. In both rows, the maximum diameter tends to align with the local orientation of the decision boundary rather than to cross it: the explanations are elongated along the boundary, i.e., they reveal directions in which the inputs can vary considerably while the network's decision stays the same.

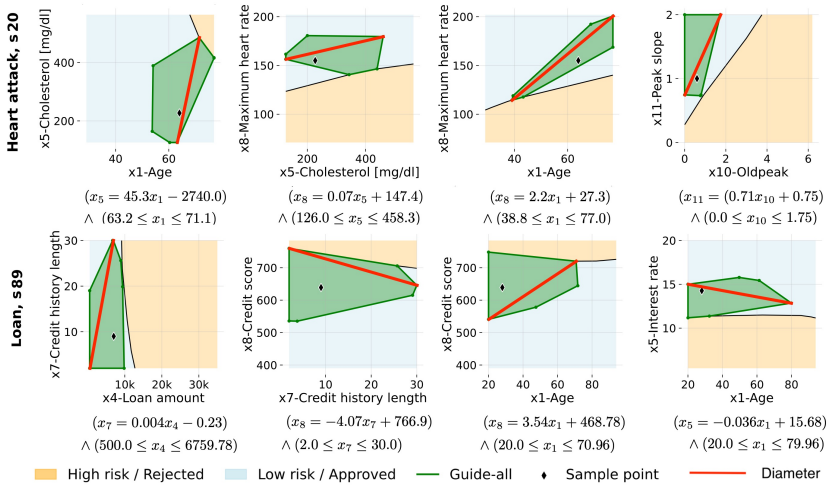


Fig. 11: Maximum diameter of explanation regions visualized in the original feature space for two datasets.

5 Conclusion

We presented SpExplAIn, a tool for computing and interpreting space explanations of neural network classifiers, which keeps the computation of an explanation separate from its interpretation. The Explanation Computation Engine produces a provably correct explanation that can capture relations between input features, and the Explanation Interpreter extracts and visualizes aspects selected by the end user, each again guaranteeing the classification, without recomputing the explanation. The tool reads FC and convolutional NNs in the `.onnx` format. It offers chainable generalization strategies, mainly `itp` with configurable interpolation algorithms; NN-specific configurations that fix or guide neuron activations, and alternative solver backends. Our evaluation on medical, financial, and image classification tasks showed that the guided configurations scale to FC networks with up to 14 hidden layers and to CNNs with about two thousand *ReLU* neurons. On tabular data, the explanations typically follow the decision boundaries and, in the inspected slices, contain those of VeriX+. While VeriX+ remains faster on wide networks with few input features, SpExplAIn is less sensitive to feature count and outperforms it on image recognition tasks.

In future work, we plan to evaluate the usability of the tool with domain experts. We further plan to extend the Explanation Interpreter to non-convex explanations, and the NN Encoder to further activation functions.

References

1. Albarghouthi, A., Dillig, I., Gurfinkel, A.: Maximal specification synthesis. SIGPLAN Not. **51**(1), 789–801 (Jan 2016), https://doi.org/10.1145/2914770.2837628

2. Alt, L., Hyvärinen, A.E.J., Sharygina, N.: LRA interpolants from no man's land. In: Strichman, O., Tzoref-Brill, R. (eds.) HVC. pp. 195–210. Springer International Publishing, Cham (2017)
3. Althoff, M.: An introduction to CORA 2015. In: Frehse, G., Althoff, M. (eds.) ARCH14-15. 1st and 2nd International Workshop on Applied veRification for Continuous and Hybrid Systems. EPiC Series in Computing, vol. 34, pp. 120–151. EasyChair (2015), https://doi.org/10.29007/zbkv
4. Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K.R., Samek, W.: On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation. PLOS One **10**(7) (2015)
5. Baehrens, D., Schroeter, T., Harmeling, S., Kawanabe, M., Hansen, K., Müller, K.R.: How to Explain Individual Classification Decisions. Journal of Machine Learning Research **11** (2010)
6. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: Fisman, D., Rosu, G. (eds.) TACAS. pp. 415–442. Springer International Publishing, Cham (2022)
7. Barrett, C., Fontaine, P., Tinelli, C.: The SMT-LIB Standard: Version 2.6. Tech. rep., Department of Computer Science, The University of Iowa (2017), https://smt-lib.org
8. Bassan, S., Elboher, Y.Y., Ladner, T., Althoff, M., Katz, G.: Explaining, fast and slow: Abstraction and refinement of provable explanations. In: Proceedings of the 42nd International Conference on Machine Learning. Proceedings of Machine Learning Research, PMLR (2025)
9. Blicha, M., Hyvärinen, A.E.J., Kofroň, J., Sharygina, N.: Decomposing Farkas interpolants. In: Vojnar, T., Zhang, L. (eds.) TACAS. pp. 3–20. Springer International Publishing, Cham (2019)
10. Bruttomesso, R., Pek, E., Sharygina, N., Tsitovich, A.: The OpenSMT Solver. In: Esparza, J., Majumdar, R. (eds.) TACAS. LNCS, vol. 6015, pp. 150–153. Springer (2010), https://doi.org/10.1007/978-3-642-12002-2_12
11. Christ, J., Hoenicke, J., Nutz, A.: SMTInterpol: An interpolating SMT solver. In: Donaldson, A.F., Parker, D. (eds.) SPIN. Lecture Notes in Computer Science, vol. 7385, pp. 248–254. Springer (2012), https://doi.org/10.1007/978-3-642-31759-0_19
12. Cimatti, A., Griggio, A., Schaafsma, B., Sebastiani, R.: The MathSAT5 SMT Solver. In: Piterman, N., Smolka, S. (eds.) TACAS. LNCS, vol. 7795, pp. 93–107. Springer (2013)
13. Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. In: J. of Symbolic Logic. pp. 269–285 (1957)
14. De Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: TACAS. pp. 337–340. TACAS'08/ETAPS'08, Springer-Verlag, Berlin, Heidelberg (2008), https://doi.org/10.1007/978-3-540-78800-3_24
15. Deng, L.: The MNIST Database of Handwritten Digit Images for Machine Learning Research. IEEE Signal Processing Magazine **29**(6), 141–142 (2012), https://doi.org/10.1109/MSP.2012.2211477
16. Dillig, I., Dillig, T., Li, B., McMillan, K.L.: Inductive invariant generation via abductive inference. In: Hosking, A.L., Eugster, P.T., Lopes, C.V. (eds.) OOPSLA. pp. 443–456. ACM (2013), https://doi.org/10.1145/2509136.2509511

17. D'Silva, V., Kroening, D., Purandare, M., Weissenbacher, G.: Interpolant strength. In: Barthe, G., Hermenegildo, M. (eds.) VMCAI. pp. 129–145. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
18. Dutertre, B.: Yices 2.2. In: Biere, A., Bloem, R. (eds.) CAV. pp. 737–744. Springer International Publishing, Cham (2014)
19. Erhan, D., Bengio, Y., Courville, A., Vincent, P.: Visualizing Higher-layer Features of a Deep Network. Tech. rep., University of Montreal (2009)
20. European Parliament and Council of the European Union: Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L series (2024)
21. Gurfinkel, A., Rollini, S.F., Sharygina, N.: Interpolation properties and SAT-based model checking. In: Van Hung, D., Ogawa, M. (eds.) ATVA. pp. 255–271. Springer International Publishing, Cham (2013)
22. Huang, X., Marques-Silva, J.: On the failings of Shapley values for explainability. International Journal of Approximate Reasoning **171** (2024)
23. Hyvärinen, A.E.J., Marescotti, M., Alt, L., Sharygina, N.: OpenSMT2: An SMT Solver for Multi-core and Cloud Computing. In: Creignou, N., Le Berre, D. (eds.) SAT. pp. 547–553. Springer International Publishing, Cham (2016)
24. Ignatiev, A., Narodytska, N., Asher, N., Marques-Silva, J.: From contrastive to abductive explanations and back again. In: AIxIA. p. 335–355. Springer-Verlag, Berlin, Heidelberg (2020), https://doi.org/10.1007/978-3-030-77091-4_21
25. Ignatiev, A., Narodytska, N., Marques-Silva, J.: Abduction-based explanations for machine learning models. In: AAAI. AAAI'19/IAAI'19/EAAI'19, AAAI Press (2019), https://doi.org/10.1609/aaai.v33i01.33011511
26. Izza, Y., Huang, X., Morgado, A., Planes, J., Ignatiev, A., Marques-Silva, J.: Distance-restricted explanations: theoretical underpinnings & efficient implementation. In: KR. KR '24 (2024), https://doi.org/10.24963/kr.2024/45
27. Izza, Y., Ignatiev, A., Stuckey, P.J., Marques-Silva, J.: Delivering inflated explanations. In: AAAI. AAAI'24/IAAI'24/EAAI'24, AAAI Press (2024), https://doi.org/10.1609/aaai.v38i11.29170
28. Izza, Y., Marques-Silva, J.: Efficient contrastive explanations on demand. CoRR **abs/2412.18262** (2024), https://doi.org/10.48550/arXiv.2412.18262
29. Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R.: Heart Disease. UCI Machine Learning Repository (1989), https://doi.org/10.24432/C52P4X
30. Junker, U.: QUICKXPLAIN: preferred explanations and relaxations for over-constrained problems. In: National Conference on Artifical Intelligence. p. 167–172. AAAI'04, AAAI Press (2004)
31. Kanav, S., Křetínský, J., Rieder, S.: A Literature Review on Verification and Abstraction of Neural Networks Within the Formal Methods Community. Springer Nature Switzerland, Cham (2025), https://doi.org/10.1007/978-3-031-75778-5_3
32. Katz, G., Huang, D.A., Ibeling, D., Julian, K., Lazarus, C., Lim, R., Shah, P., Thakoor, S., Wu, H., Zeljić, A., Dill, D.L., Kochenderfer, M.J., Barrett, C.: The Marabou framework for verification and analysis of deep neural networks. In: Dillig, I., Tasiran, S. (eds.) CAV. pp. 443–452. Springer International Publishing, Cham (2019)
33. Kolárik, T., Fedyukovich, G., Labbaf, F., Leopardi, F., Sharygina, N., Wand, M.: Formally explaining neural network classification. In: Sampaio, A., Stoelinga, M. (eds.) Formal Methods. pp. 471–496. Springer Nature Switzerland, Cham (2026)

34. Kolárik, T., Labbaf, F., Leopardi, F., Fedyukovich, G., Wand, M., Sharygina, N.: Neuron activation-based computation of logical explanations for deep neural networks. In: ATVA (2026), extended version at https://arxiv.org/abs/2609.14099
35. König, M., Bosman, A.W., Hoos, H.H., Van Rijn, J.N.: Critically assessing the state of the art in neural network verification. J. Mach. Learn. Res. **25**(1) (Jan 2024)
36. Krizhevsky, A.: Learning multiple layers of features from tiny images. Tech. rep., University of Toronto (2009)
37. Labbaf, F., Kolárik, T., Blicha, M., Fedyukovich, G., Wand, M., Sharygina, N.: Space explanations of neural network classification. In: Piskac, R., Rakamaric, Z. (eds.) CAV. Springer International Publishing (2025)
38. Leopardi, F., Labbaf, F., Kolárik, T., Wand, M., Sharygina, N.: Interpreting logical explanations of classifying neural networks. In: ESANN (2026)
39. Lo, T.w.: Loan approval classification dataset. https://www.kaggle.com/datasets/taweilo/loan-approval-classification-data (2024), version of 29 October 2024. Accessed YYYY-MM-DD
40. Lundberg, S.M., Lee, S.I.: A Unified Approach to Interpreting Model Predictions. In: NIPS (2017)
41. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. In: ICLR. OpenReview.net (2018), https://openreview.net/forum?id=rJzIBfZAb
42. Mahendran, A., Vedaldi, A.: Understanding Deep Image Representations by Inverting Them. In: CVPR. pp. 5188 – 5196 (2015)
43. Malfa, E.L., Michelmore, R., Zbrzezny, A.M., Paoletti, N., Kwiatkowska, M.: On guaranteed optimal robust explanations for NLP models. In: IJCAI. pp. 2658 – 2665 (2021)
44. Marques-Silva, J.: Logic-Based Explainability in Machine Learning. Springer Nature Switzerland, Cham (2023), https://doi.org/10.1007/978-3-031-31414-8_2
45. McMillan, K.L.: Interpolation and SAT-based model checking. In: Hunt, W.A., Somenzi, F. (eds.) CAV. pp. 1–13. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
46. de Moura, L., Bjørner, N.: Satisfiability Modulo Theories: Introduction and Applications. Communications of the ACM **54**(9), 69 – 77 (2011)
47. Nieuwenhuis, R., Oliveras, A., Tinelli, C.: Solving SAT and SAT modulo theories: from an abstract Davis–Putnam–Logemann-Loveland procedure to DPLL(T). Journal of the ACM (JACM) **53**(6), 937–977 (2006), https://doi.org/10.1145/1217856.1217859
48. Nocedal, J., Wright, S.J.: Quadratic programming. In: Numerical Optimization. Springer New York, New York, NY, 2 edn. (2006)
49. Palechor, F.M., Manotas, A.D.l.H.: Estimation of Obesity Levels Based On Eating Habits and Physical Condition. UCI Machine Learning Repository (2019), https://doi.org/10.24432/C5H31Z
50. Prabhu, S., D'Souza, D., Chakraborty, S., Venkatesh, R., Fedyukovich, G.: Weakest precondition inference for non-deterministic linear array programs. In: Finkbeiner, B., Kovács, L. (eds.) TACAS, Part II. Lecture Notes in Computer Science, vol. 14571, pp. 175–195. Springer (2024), https://doi.org/10.1007/978-3-031-57249-4_9
51. Prabhu, S., Fedyukovich, G., D'Souza, D.: Maximal quantified precondition synthesis for linear array loops. In: Weirich, S. (ed.) ESOP, Part II. Lecture Notes

in Computer Science, vol. 14577, pp. 245–274. Springer (2024), https://doi.org/10.1007/978-3-031-57267-8_10
52. Ribeiro, M.T., Singh, S., Guestrin, C.: "Why Should I Trust You?": Explaining the predictions of any classifier. In: KDD. p. 1135–1144. KDD '16, Association for Computing Machinery, New York, NY, USA (2016), https://doi.org/10.1145/2939672.2939778
53. Ribeiro, M.T., Singh, S., Guestrin, C.: Anchors: High-Precision Model-Agnostic Explanations. In: AAAI (2018)
54. Rollini, S.F., Sery, O., Sharygina, N.: Leveraging interpolant strength in model checking. In: Madhusudan, P., Seshia, S.A. (eds.) Computer Aided Verification. pp. 193–209. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
55. Seshia, S.A., Sadigh, D., Sastry, S.S.: Towards Verified Artificial Intelligence. Communications of the ACM **65**(7), 46 – 55 (2022)
56. Shih, A., Choi, A., Darwiche, A.: A Symbolic Approach to Explaining Bayesian Network Classifiers. In: IJCAI. pp. 5103 – 5111 (2018)
57. Simonyan, K., Vedaldi, A., Zisserman, A.: Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. In: Workshop Proceedings of the ICLR (2014)
58. Slack, D., Hilgard, S., Jia, E., Singh, S., Lakkaraju, H.: Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods. In: AIES. pp. 180–186. ACM (2020)
59. Stallkamp, J., Schlipsing, M., Salmen, J., Igel, C.: The German Traffic Sign Recognition Benchmark: A multi-class classification competition. In: IEEE International Joint Conference on Neural Networks. pp. 1453–1460 (2011)
60. Wu, M., Li, X., Wu, H., Barrett, C.: Efficiently computing compact formal explanations. Proceedings of the AAAI Conference on Artificial Intelligence **40**(42), 35857–35866 (Mar 2026), https://doi.org/10.1609/aaai.v40i42.40900
61. Wu, M., Wu, H., Barrett, C.: VeriX: Towards verified explainability of deep neural networks. In: Oh, A., Neumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems. vol. 36, pp. 22247–22268. Curran Associates, Inc. (2023), https://proceedings.neurips.cc/paper_files/paper/2023/file/46907c2ff9fafd618095161d76461842-Paper-Conference.pdf
62. Zeiler, M.D., Fergus, R.: Visualizing and Understanding Convolutional Networks. In: ECCV. pp. 818 – 833 (2014)

A CNN Architectures

Table 1 summarizes the benchmark CNNs, and Tables 2–5 list them layer by layer. For example, S3 is c8s2 → c16s2 → c16 → FC_{32} → FC_K; on MNIST, its feature maps are 8×14×14, 16×7×7, and 16×7×7, giving 1568+784+784+32 = 3168 *ReLU* neurons.

Table 1: CNN architectures from input to output, with channel counts of the wide models (halved in the narrow N-variants). ck: 3×3 convolution with k channels, stride 1, padding 1 (keeps the size); cks2: 4×4 convolution with k channels, stride 2, padding 1 (halves the size); a: 2×2 average pooling, stride 2 (halves the size). Each convolution is followed by a *ReLU*, and each network ends with FC_{32}+*ReLU* → FC_K (K = number of classes). #*ReLU*: number of *ReLU* neurons (wide/narrow), equal for Sd and ABd. Inputs: 1×28^2 (MNIST), 3×32^2 (GTSRB, CIFAR10).

	Sd / NSd	ABd / NABd	#*ReLU* (wide/narrow)	
d	(strided)	(avg. pooling)	MNIST	GTSRB CIFAR10
1	c8s2	c8s2 a	1600/816	2080/1056
2	c8s2 c16s2	c8s2 a c16	2384/1208	3104/1568
3	c8s2 c16s2 c16	c8s2 a c16 c16 a	3168/1600	4128/2080
4	c8s2 c8 c8s2 c8	c8s2 c8 a c8 c8 a	3952/1992	5152/2592
5	c8s2 c8 c8s2 c16 c32s2	c8s2 c8 a c8 c16 a c32 a	4632/2332	6176/3104
6	c8s2 c8 c16s2 c16 c32s2 c32	c8s2 c8 a c16 c16 a c32 c32 a	5312/2672	7200/3616

Table 2: Layer-by-layer architecture of the benchmark CNNs on MNIST, $d = 1, \ldots, 3$ convolutions (left: stride-2 downsampling; right: average pooling). Layers use the notation of Table 1. Each convolution and `fc32` (fully connected, 32 neurons) is followed by a ReLU; `fc32` takes the flattened feature map, and the final linear layer (one logit per class) is omitted. Layer names and shapes ($C\times H\times W$) are for the wide model; the narrow twin (`N` prefix) has the same spatial sizes with half the channels. ReLUs is the number of ReLU neurons of the layer, wide/narrow; `a` has none. GTSRB and CIFAR-10 have identical layer shapes.

S1 / **NS1**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`fc32`	1568	32	32/32
total			1600/816

AB1 / **NAB1**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`a`	8×14×14	8×7×7	–
`fc32`	392	32	32/32
total			1600/816

S2 / **NS2**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c16s2`	8×14×14	16×7×7	784/392
`fc32`	784	32	32/32
total			2384/1208

AB2 / **NAB2**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`a`	8×14×14	8×7×7	–
`c16`	8×7×7	16×7×7	784/392
`fc32`	784	32	32/32
total			2384/1208

S3 / **NS3**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c16s2`	8×14×14	16×7×7	784/392
`c16`	16×7×7	16×7×7	784/392
`fc32`	784	32	32/32
total			3168/1600

AB3 / **NAB3**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`a`	8×14×14	8×7×7	–
`c16`	8×7×7	16×7×7	784/392
`c16`	16×7×7	16×7×7	784/392
`a`	16×7×7	16×3×3	–
`fc32`	144	32	32/32
total			3168/1600

Table 3: Layer-by-layer architecture of the benchmark CNNs on MNIST, $d = 4, \ldots, 6$ convolutions (left: stride-2 downsampling; right: average pooling). Notation as in Table 2.

S4 / **NS4**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c8`	8×14×14	8×14×14	1568/784
`c8s2`	8×14×14	8×7×7	392/196
`c8`	8×7×7	8×7×7	392/196
`fc32`	392	32	32/32
total			3952/1992

AB4 / **NAB4**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c8`	8×14×14	8×14×14	1568/784
`a`	8×14×14	8×7×7	–
`c8`	8×7×7	8×7×7	392/196
`c8`	8×7×7	8×7×7	392/196
`a`	8×7×7	8×3×3	–
`fc32`	72	32	32/32
total			3952/1992

S5 / **NS5**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c8`	8×14×14	8×14×14	1568/784
`c8s2`	8×14×14	8×7×7	392/196
`c16`	8×7×7	16×7×7	784/392
`c32s2`	16×7×7	32×3×3	288/144
`fc32`	288	32	32/32
total			4632/2332

AB5 / **NAB5**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c8`	8×14×14	8×14×14	1568/784
`a`	8×14×14	8×7×7	–
`c8`	8×7×7	8×7×7	392/196
`c16`	8×7×7	16×7×7	784/392
`a`	16×7×7	16×3×3	–
`c32`	16×3×3	32×3×3	288/144
`a`	32×3×3	32×1×1	–
`fc32`	32	32	32/32
total			4632/2332

S6 / **NS6**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c8`	8×14×14	8×14×14	1568/784
`c16s2`	8×14×14	16×7×7	784/392
`c16`	16×7×7	16×7×7	784/392
`c32s2`	16×7×7	32×3×3	288/144
`c32`	32×3×3	32×3×3	288/144
`fc32`	288	32	32/32
total			5312/2672

AB6 / **NAB6**

Layer	Input	Output	ReLUs
`c8s2`	1×28×28	8×14×14	1568/784
`c8`	8×14×14	8×14×14	1568/784
`a`	8×14×14	8×7×7	–
`c16`	8×7×7	16×7×7	784/392
`c16`	16×7×7	16×7×7	784/392
`a`	16×7×7	16×3×3	–
`c32`	16×3×3	32×3×3	288/144
`c32`	32×3×3	32×3×3	288/144
`a`	32×3×3	32×1×1	–
`fc32`	32	32	32/32
total			5312/2672

Table 4: Layer-by-layer architecture of the benchmark CNNs on GTSRB, CIFAR, $d = 1, \ldots, 3$ convolutions (left: stride-2 downsampling; right: average pooling). Notation as in Table 2.

S1 / **NS1**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`fc32`	2048	32	32/32
total			2080/1056

S2 / **NS2**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c16s2`	8×16×16	16×8×8	1024/512
`fc32`	1024	32	32/32
total			3104/1568

S3 / **NS3**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c16s2`	8×16×16	16×8×8	1024/512
`c16`	16×8×8	16×8×8	1024/512
`fc32`	1024	32	32/32
total			4128/2080

AB1 / **NAB1**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`a`	8×16×16	8×8×8	–
`fc32`	512	32	32/32
total			2080/1056

AB2 / **NAB2**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`a`	8×16×16	8×8×8	–
`c16`	8×8×8	16×8×8	1024/512
`fc32`	1024	32	32/32
total			3104/1568

AB3 / **NAB3**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`a`	8×16×16	8×8×8	–
`c16`	8×8×8	16×8×8	1024/512
`c16`	16×8×8	16×8×8	1024/512
`a`	16×8×8	16×4×4	–
`fc32`	256	32	32/32
total			4128/2080

Table 5: Layer-by-layer architecture of the benchmark CNNs on GTSRB, CIFAR, $d = 4, \ldots, 6$ convolutions (left: stride-2 downsampling; right: average pooling). Notation as in Table 2.

S4 / **NS4**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c8`	8×16×16	8×16×16	2048/1024
`c8s2`	8×16×16	8×8×8	512/256
`c8`	8×8×8	8×8×8	512/256
`fc32`	512	32	32/32
total			5152/2592

AB4 / **NAB4**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c8`	8×16×16	8×16×16	2048/1024
`a`	8×16×16	8×8×8	–
`c8`	8×8×8	8×8×8	512/256
`c8`	8×8×8	8×8×8	512/256
`a`	8×8×8	8×4×4	–
`fc32`	128	32	32/32
total			5152/2592

S5 / **NS5**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c8`	8×16×16	8×16×16	2048/1024
`c8s2`	8×16×16	8×8×8	512/256
`c16`	8×8×8	16×8×8	1024/512
`c32s2`	16×8×8	32×4×4	512/256
`fc32`	512	32	32/32
total			6176/3104

AB5 / **NAB5**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c8`	8×16×16	8×16×16	2048/1024
`a`	8×16×16	8×8×8	–
`c8`	8×8×8	8×8×8	512/256
`c16`	8×8×8	16×8×8	1024/512
`a`	16×8×8	16×4×4	–
`c32`	16×4×4	32×4×4	512/256
`a`	32×4×4	32×2×2	–
`fc32`	128	32	32/32
total			6176/3104

S6 / **NS6**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c8`	8×16×16	8×16×16	2048/1024
`c16s2`	8×16×16	16×8×8	1024/512
`c16`	16×8×8	16×8×8	1024/512
`c32s2`	16×8×8	32×4×4	512/256
`c32`	32×4×4	32×4×4	512/256
`fc32`	512	32	32/32
total			7200/3616

AB6 / **NAB6**

Layer	Input	Output	ReLUs
`c8s2`	3×32×32	8×16×16	2048/1024
`c8`	8×16×16	8×16×16	2048/1024
`a`	8×16×16	8×8×8	–
`c16`	8×8×8	16×8×8	1024/512
`c16`	16×8×8	16×8×8	1024/512
`a`	16×8×8	16×4×4	–
`c32`	16×4×4	32×4×4	512/256
`c32`	32×4×4	32×4×4	512/256
`a`	32×4×4	32×2×2	–
`fc32`	128	32	32/32
total			7200/3616

B Additional Plots

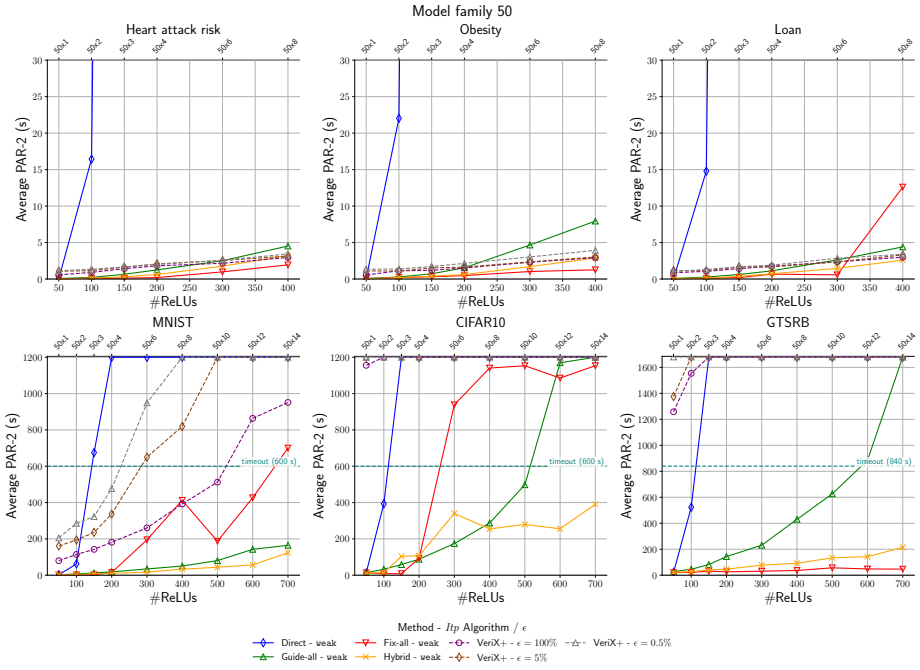
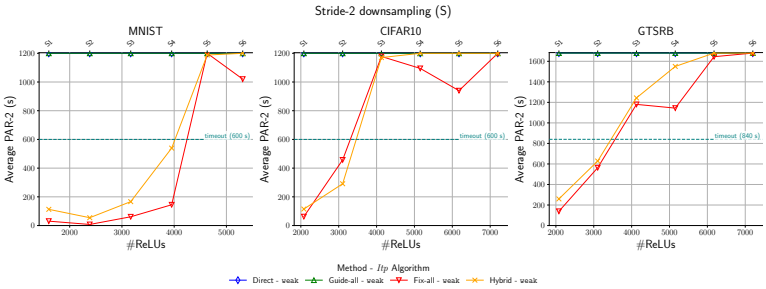
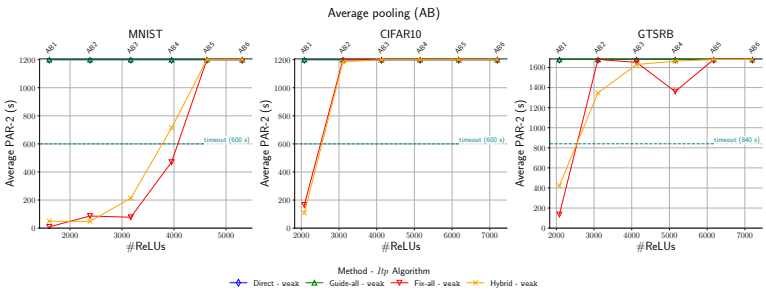


Fig. 12: PAR-2 runtime (s) of SpExplAIn (**weak**) and VeriX+ ($\epsilon \in \{0.5\%, 5\%, 100\%\}$) w.r.t. the depth of the FC networks with 50 neurons per layer.



(a) Wide, stride-2 downsampling (S).



(b) Wide, average pooling (AB).

Fig. 13: PAR-2 runtime (s) of our configurations with `weak` interpolation w.r.t. $\#ReLU$s and depth of wide CNNs.